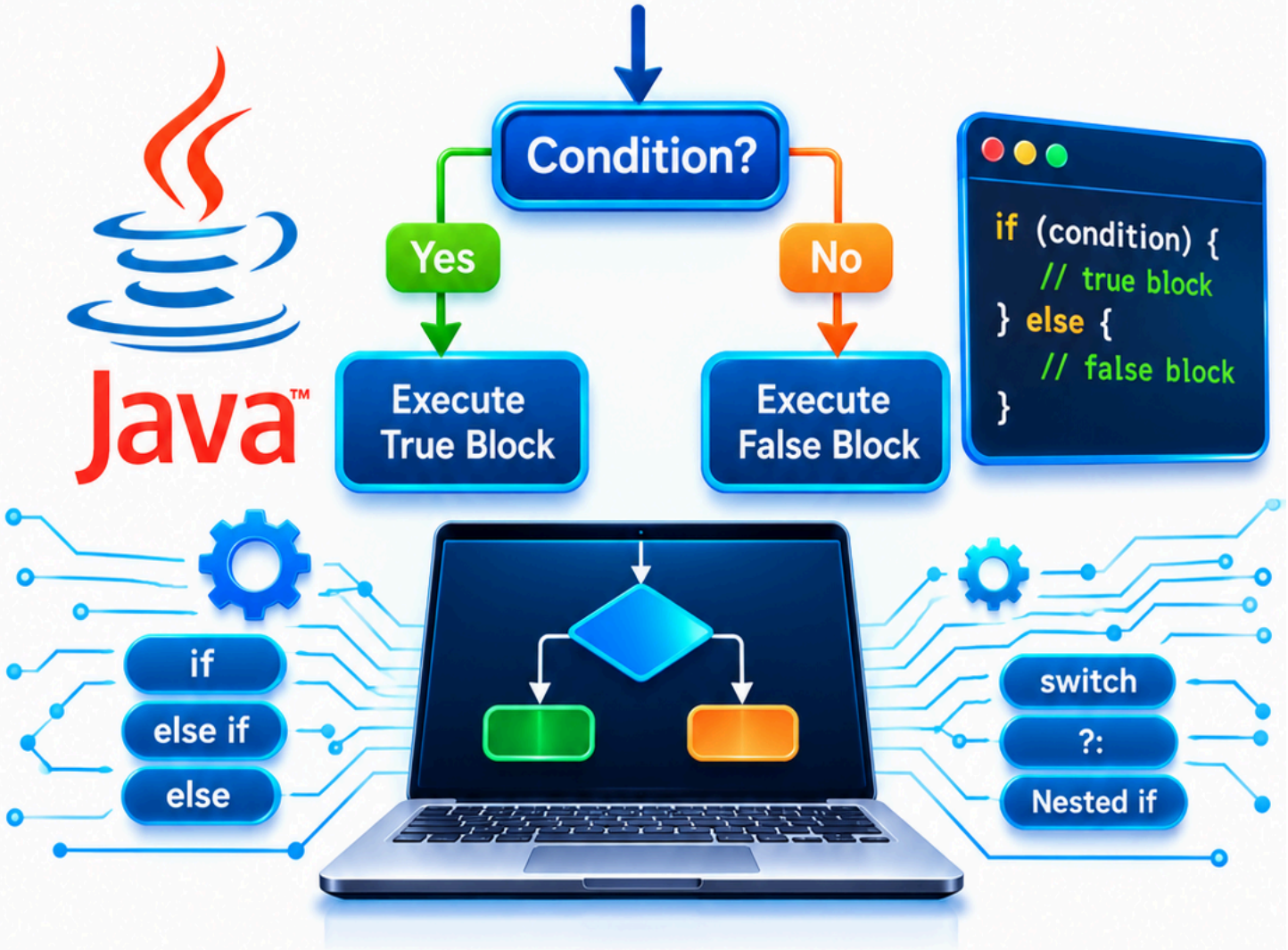
# CHAPTER 5

# DECISION MAKING AND BRANCHING IN JAVA



# INDEX


| SL. No. | Topic | Page No. |
|---|---|---|
| 1 | Fundamentals of Decision Making | 3 |
| 2 | The Simple if Statement | 4 |
| 3 | The If…Else Statement | 6 |
| 4 | The Nested-If Statement | 8 |
| 5 | ElseIf Ladder | 10 |
| 6 | The Switch Statement | 12 |
| 7 | If Vs Switch | 16 |


ABHILASH B J

# DECISION MAKING AND BRANCING

## Executive Summary

Conditional execution forms the fundamental basis of program design, enabling software applications to evaluate incoming data against specified criteria and dynamically route control flow. In Java programming, decision-making constructs allow developers to control which code blocks are executed based on the boolean evaluation (`true` or `false`) of test conditions.

This briefing document details the core concepts of decision making and branching, focusing specifically on the structural mechanics, syntax, execution flows, and implementation examples of **Simple If** statements and **If...Else** statements.

## Fundamentals of Decision Making

When building applications, developers must control the execution of specific code sections based on the data being processed—a technique known as **conditionally executing program code**.

### Primary Mechanics

- **Condition / Test Evaluation:** The program evaluates a specified test or condition.
- **Control Routing:** If the condition evaluates to `true`, the corresponding block of code is processed. If `false`, the execution bypasses the block or diverts to an alternative path.
- **Core Role:** Conditional logic is considered the basis of every program that is written.

### Java Decision-Making Categories

Java supports three primary decision-making constructs:

1. **`if` statements**
2. **`switch` statement**
3. **Conditional operator statement**

ABHILASH B J

## Classification of `if` Statements

An `if` statement evaluates an expression first. Depending on whether the expression yields a value of `true` or `false`, specific statements are executed.

Java supports four distinct variants of `if` statements:

1. **Simple `if` statement**
2. **`If..else` statement**
3. **Nested `if` statement**
4. **`If..else if` statement**

## Structural Analysis & Programming Implementation

## The Simple `if` Statement

The simple `if` statement tests a basic condition.

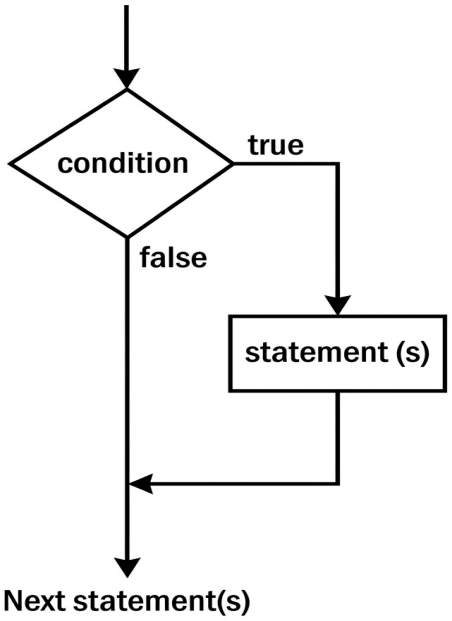
- If the `condition` is **true**, the statement block inside the curly braces is executed.
- If the `condition` is **false**, the statement block is entirely skipped, and execution jumps directly to the statement immediately following the block.
- A statement block may consist of a single statement or a group of statements.

#### Syntax Definition

```java
if (condition)
{
    .......
    ....... (statement block)
    .......
}
```

HILASH B J

**Flowchart:**




**Code Implementation Example: Simple `if`**

```java
// Largest of two numbers
class largest
{
   public static void main(String arg[])
   {
      int a,b;
      a=20;
      b=10;

      if(a>b)
          System.out.println("A is Largest Number");

      if(b>a)
          System.out.println("B is Largest Number");
   }
}
```

**Program Output:**

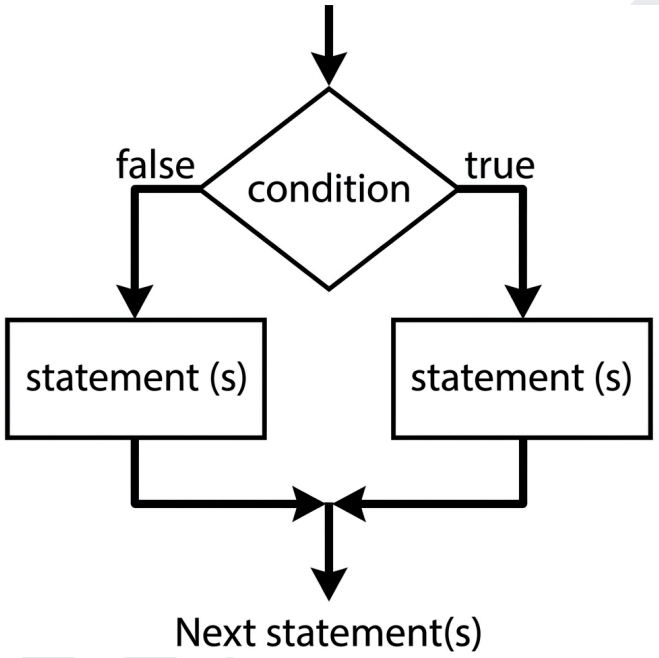
```
A is Largest Number
```

## The `If...Else` Statement

The `if...else` statement expands upon the simple `if` construct by providing an explicit execution path for both outcomes of a condition.

- If the `condition` evaluates to **true**, the **true statement block** is executed.
- If the `condition` evaluates to **false**, the **false statement block** is executed.
- Under no circumstances are both statement blocks executed; control merges at the next statement after the entire construct completes.

**Flowchart:**




**Syntax Definition**

```java
if (condition)
{
   .......
   ....... ( true statement block)
   .......
}
else
{
   .......
   ....... ( false statement block)
   .......
}
```

**Code Implementation Example: `If...Else`**

```java
// Largest of two numbers
class largest
{
   public static void main(String arg[])
   {
      int a,b;
      a=20;
      b=10;

      if(a>b)
          System.out.println("A is Largest Number");
      else
          System.out.println("B is Largest Number");
   }
}
```

**Program Output:**

```
A is Largest Number
```

## Key Comparative Summary

| Construct | Condition Evaluation (`true`) | Condition Evaluation (`false`) | Typical Use Case |
|---|---|---|---|
| **Simple `if`** | Executes the enclosed statement block, then moves to next statement. | Skips the statement block and jumps directly to next statement. | Optional execution of a specific code block when a criteria is met. |
| **`If...Else`** | Executes the `true statement block`, bypassing the `else` block. | Skips the `true` block and executes the `false statement block`. | Binary branching where two mutually exclusive actions must be handled. |

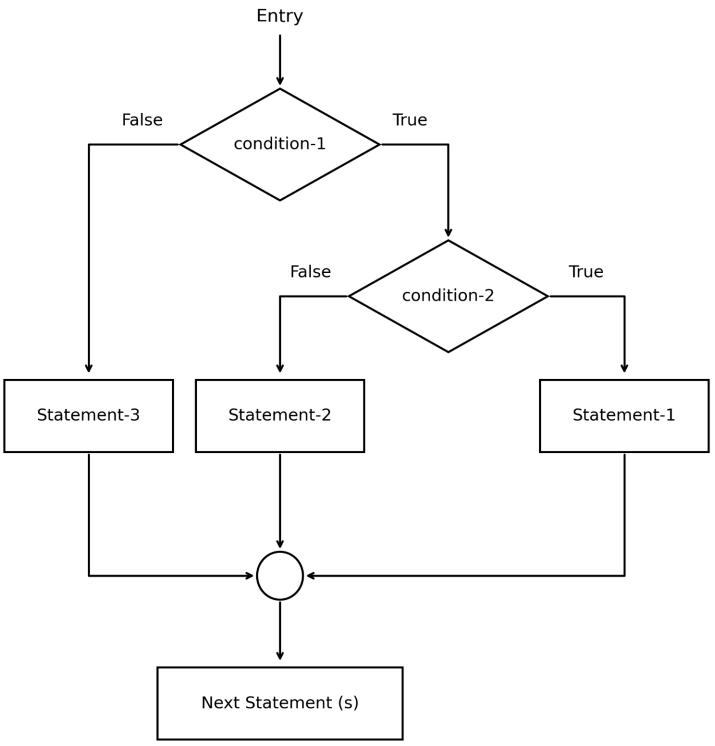
## The Nested-If Statement

A nested-if statement consists of an `if` statement placed inside the body of another `if` or `else` statement.

**Syntax:**

```java
if (condition-1)
{
    if (condition-2)
    {
        ..... (statement block-1)
    }
    else
    {
        ..... (statement block-2)
    }
}
else
{
    ..... (statement block-3)
}
```

**Flowchart:**



**Control Flow Rules:**

1. If `condition-1` is **true**, the program evaluates `condition-2`.
    - If `condition-2` is **true**, `statement block-1` executes.
    - If `condition-2` is **false**, `statement block-2` executes.
    - In both cases, control is subsequently transferred to the statement following the outer `if` construct.
2. If `condition-1` is **false**, `statement block-3` executes directly, bypassing `condition-2` and its associated blocks completely.

### Code Example: Finding the Largest of Three Numbers

Java

```java
// Largest of Three numbers
class largest
{
    public static void main(String arg[])
    {
        int a,b,c;
        a=20;
        b=10;
        c=30;
        if(a>b)
        {
            if (a>c)
                System.out.println("A is Largest Number");
        }
        else
        {
            if (b>c)
                System.out.println("B is Largest Number");
            else
                System.out.println("C is Largest Number");
        }
    }
}
```

**Output:**

Java

```
C is Largest Number
```

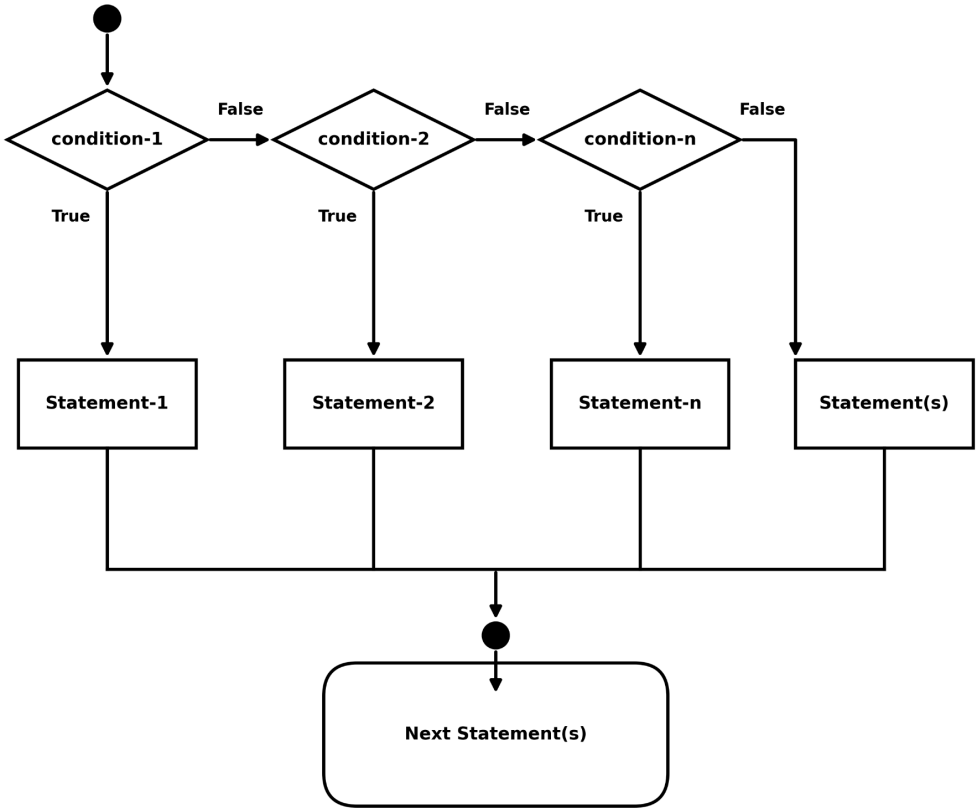
## The If...Else If...Else Statement (Elseif Ladder)

The `if...else if...else` structure (commonly known as the elseif ladder) evaluates conditions in a top-down chain.

**Syntax:**

```java
if (condition-1)
{
    ......
    ...... (statement block-1)
    ......
}
else if (condition-2)
{
    ......
    ...... (statement block-2)
    ......
}
....
....
else if (condition-N)
{
    ......
    ...... (statement block-n)
    ......
}
else
{
    ......
    ...... (default statement block)
    ......
}
```

**Flowchart:**




**Control Flow Rules:**

- Conditions are evaluated starting from the top (`condition-1`).
- As soon as a **true** condition is encountered, its corresponding statement block executes, and control immediately transfers to the next statement outside the entire `if` construct.
- If all defined conditions evaluate to **false**, the `default statement block` under the final `else` clause executes.

## Code Example: Student Grade Calculation

Java

```java
// To find the grade of a student
class grade
{
    public static void main(String arg[])
    {
        double average=85;
        if(average>=80)
            System.out.println("Honours");
```

```
        else if (average>=60 && average<80)
            System.out.println("First Division");
        else if (average>=50 && average<60)
            System.out.println("Second Division");
        else if (average>=50 && average<60)
            System.out.println("Second Division");
        else if (average>=40 && average<50)
            System.out.println("Third Division");
        else
            System.out.println("Fail");
    }
}
```

**Output:**

```
None
Honours
```

## The Switch Statement

The `switch` statement is Java's built-in multi-way decision structure. It matches an expression against a list of predefined constant `case` values.

**Syntax:**

```
Java
switch (expression)
{
    case value-1:
        ..... (statement block-1)
        break;
    case value-2:
        ..... (statement block-2)
        break;
```

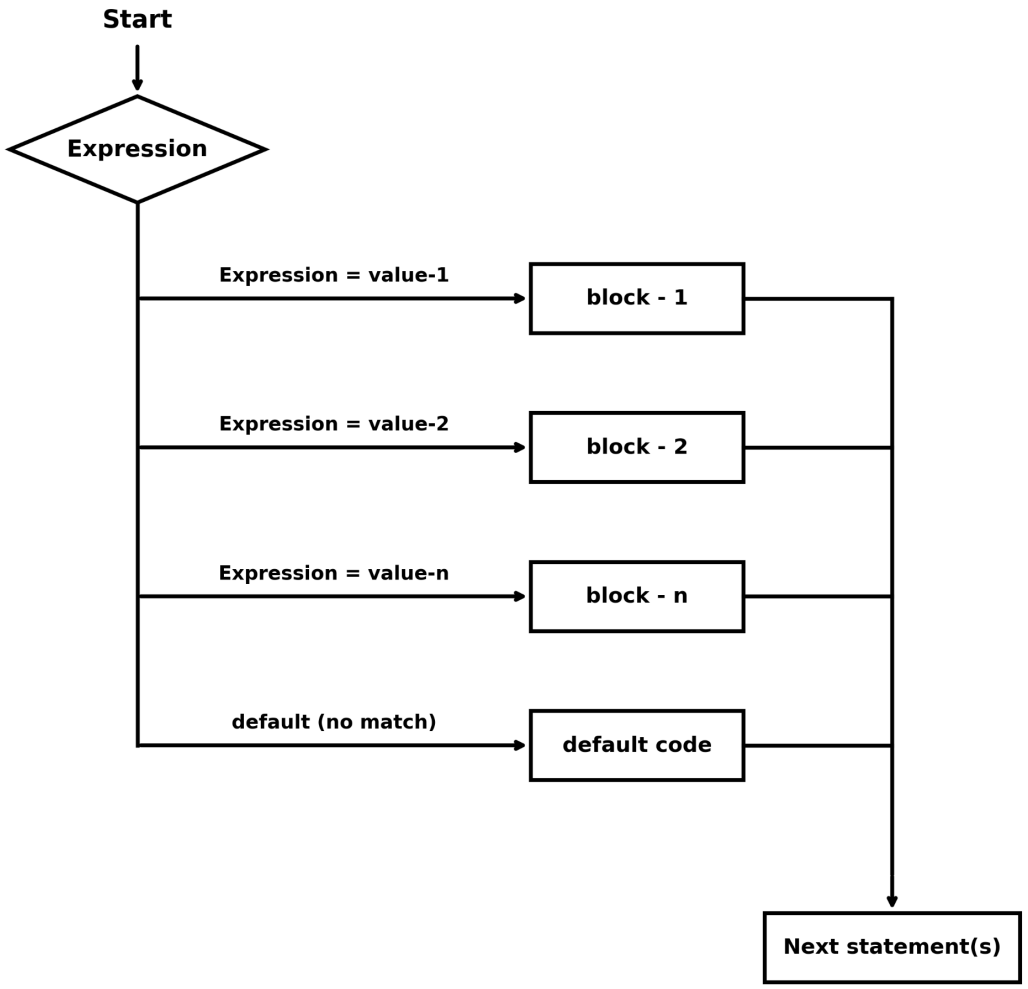
```
    .....
    .....
    case value-n:
        ...... (statement block-n)
        break;
    default:
        ...... (default statement block)
        break;
}
```

**Operational Rules:**

1. **Permitted Data Types**: The variable or expression used inside a `switch` statement can only be integers, convertible integers (`byte`, `short`, `char`), or `String` types.
2. **Case Definitions**: A `switch` block can contain any number of `case` statements. Each case is followed by the comparison value and a colon (`:`).
3. **Value Characteristics**: The value provided for a `case` must match the data type of the `switch` variable and must be a literal or constant.
4. **Execution Flow**: When the evaluated variable equals a `case` value, the statements following that `case` execute sequentially until a `break` statement is reached.
5. **Break Behavior**: Upon encountering a `break` statement, the `switch` block terminates immediately, jumping to the statement following the switch block.
6. **Fall-Through Execution**: If a `case` block does not end with a `break`, execution continues ("falls through") into subsequent `case` blocks regardless of whether those cases match, until a `break` or the end of the `switch` is reached.
7. **Default Case**: The optional `default` case executes if none of the explicit `case` values match the expression.

ABHILASH B

**Flowchart:**




## Code Example: Menu-Driven Arithmetic Operations

Java

```
// Arithmetic operations using switch
import java.io.*;
class arithmetic
{
    public static void main(String arg[]) throws IOException
    {
        int a,b,c,ch;
        InputStreamReader read=new InputStreamReader(System.in);
        BufferedReader in=new BufferedReader(read);
        System.out.println("1.Addition 2. Subtraction
3.Multiplication 4.Division");
        System.out.println("Enter Your choice....");
        ch=Integer.parseInt(in.readLine());
        System.out.println ("Enter two numbers");
```

```
        a=Integer.parseInt(in.readLine());
        b=Integer.parseInt(in.readLine());
        switch(ch)
        {
            case 1:
                c=a+b;
                System.out.println("Addition="+c);
                break;
            case 2:
                c=a-b;
                System.out.println("Subtraction="+c);
                break;
            case 3:
                c=a*b;
                System.out.println("Multiplication="+c);
                break;
            case 4:
                c=a/b;
                System.out.println("Division="+c);
                break;
            default:
                System.out.println("Invalid choice");
        }
    }
}
```

**Sample Execution Output:**

```
Java
1.Addition 2. Subtraction 3.Multiplication 4.Division
Enter Your choice....
1
Enter two numbers
10
20
Addition=30
```

## Difference Between `if` and `switch` Statements

The table below summarizes the architectural and operational differences between `if` conditional statements and `switch`branching statements:

| S.No. | `if` Statement | `switch` Statement |
|---|---|---|
| 1 | Expression may be of any data type. | Case values must be an `integer` or `char` type (or convertible integer/string). |
| 2 | Makes a decision based on binary `true` or `false` evaluations. | Makes a decision from a discrete number of choices. |
| 3 | Does not allow `break` statements by default. | Allows and utilizes `break` statements by default to control path termination. |
| 4 | Useful for writing top-down programming constructs. | Useful for writing menu-driven programming constructs. |
| 5 | The `default` (`else`) statement must appear at the end of the construct. | The `default` case may be placed anywhere within the `switch` block. |

ABHILASH B J

# THANK YOU

ABHILASH B J